

GUIA_DEPLOYMENT_PASSO_A_PASSO

Guia completo para publicar a solução **Cidades Verdes (API Ambiente + Firebird)** em produção.

- **Código fonte local (origem):** `/home/ubuntu/cidadesverdes`
- **Servidor de produção (destino):** `/home/deploy/var/www/sistema-gestao/cidadesverdes`
- **Usuário de produção:** `deploy`
- **Porta da API Ambiente:** `8101`
- **Banco Firebird:** `/home/deploy/var/databases/cidadesverdes.fdb`

Pré-requisitos assumidos: Git já configurado no servidor, Python 3.8.10 e `.venv` já existentes, UFW ativo.

PASSO 1: Preparar arquivos para commit no GitHub

O que será feito

Você vai copiar os arquivos novos/alterados da solução local (`/home/ubuntu/cidadesverdes`) para o seu repositório Git local de trabalho, revisar diferenças e fazer commit/push.

Comandos (copiar e colar)

Execute na máquina de desenvolvimento onde está o clone do repositório que sobe para produção.

```
# 1) Definir caminhos
SOURCE_DIR="/home/ubuntu/cidadesverdes"
TARGET_REPO="/caminho/do/seu/repo-git" # ajuste este caminho

# 2) Entrar no repo existente
cd "$TARGET_REPO"

# 3) (Opcional, recomendado) Criar branch de release/deploy
git checkout -b chore/deploy-api-ambiente-8101

# 4) Copiar arquivos mantendo estrutura
rsync -av --delete \
  --exclude='.git' \
  --exclude='.venv' \
  --exclude='__pycache__' \
  --exclude='*.pyc' \
  "$SOURCE_DIR/" "$TARGET_REPO/"

# 5) Verificar o que mudou
git status
git diff --stat

# 6) (Opcional) Revisar conteúdo exato alterado
git diff

# 7) Preparar commit
git add .
git status

# 8) Commit
git commit -m "feat: adiciona API ambiente (8101) com Firebird e automação de deploy"

# 9) Push da branch
git push -u origin chore/deploy-api-ambiente-8101
```

Output esperado

- `rsync` : lista de arquivos copiados.
- `git status` : arquivos em `Changes to be committed`.
- `git commit` : mensagem com quantidade de arquivos alterados.
- `git push` : branch remota criada/atualizada com sucesso.

Verificação

```
git log --oneline -n 3
git branch -vv
```

Troubleshooting comum

- **fatal: not a git repository** : você não está dentro do clone correto (`cd "$TARGET_REPO"`).
- **Permission denied no push** : validar chave SSH/token GitHub.
- **Arquivos indevidos no commit** : revisar `.gitignore` e rodar `git reset` + `git add` seletivo.

PASSO 2: Atualizar o servidor via Git Pull

O que será feito

Conectar no servidor, entrar no diretório da aplicação e trazer as alterações do GitHub.

Comandos (copiar e colar)

```
# 1) Acessar servidor
ssh deploy@SEU_SERVIDOR

# 2) Ir para projeto
cd /home/deploy/var/www/sistema-gestao/cidadesverdes

# 3) Verificar branch atual
git branch --show-current

# 4) Atualizar referências remotas
git fetch --all --prune

# 5) Se for deploy direto na branch principal (ex.: main)
git pull origin main

# 6) Se você usa branch específica de release
git checkout chore/deploy-api-ambiente-8101
git pull origin chore/deploy-api-ambiente-8101
```

Output esperado

- `git pull` : Already up to date. **ou** lista de arquivos atualizados.

Verificação

```
git log --oneline -n 5
ls -la app_ambiente scripts systemd sql
```

Troubleshooting comum

- **Conflito no pull**: resolver conflito, commitar e repetir pull.
- **Branch não encontrada no servidor**: rode `git fetch --all --prune` antes de `git checkout`.

PASSO 3: Instalar Firebird 3.0.13

O que será feito

Instalar (ou validar) Firebird 3.0 no Ubuntu/Debian, iniciar serviço e conferir versão.

Comandos (copiar e colar)

```
cd /home/deploy/var/www/sistema-gestao/cidadesverdes
chmod +x scripts/setup_firebird.sh
./scripts/setup_firebird.sh
```

Verificação detalhada

```
# Versão instalada
dpkg-query -W -f='${Version}\n' firebird3.0-server

# Serviço ativo
sudo systemctl status firebird3.0 --no-pager

# Porta padrão 3050 escutando
sudo ss -ltnp | grep 3050 || true
```

Configuração inicial do Firebird

```
# Ver senha atual do SYSDBA (gerada/definida no pacote Debian/Ubuntu)
sudo cat /etc/firebird/3.0/SYSDBA.password

# (Opcional) alterar senha SYSDBA para uma senha forte
sudo -u firebird gsec -user sysdba -password masterkey -modify sysdba -pw 'NOVA_SENHA_FORTE_AQUI'
```

Output esperado

- Serviço `firebird3.0` em estado `active (running)`.
- Arquivo `/etc/firebird/3.0/SYSDBA.password` disponível.

Troubleshooting comum

- **Versão não exatamente 3.0.13:** em alguns repositórios pode vir 3.0.x; validar compatibilidade e seguir.
- **Serviço não inicia:** verificar `journalctl -u firebird3.0 -n 100 --no-pager`.

PASSO 4: Criar diretório para o banco de dados

O que será feito

Criar diretório de dados do Firebird e ajustar permissões para leitura/escrita.

Comandos (copiar e colar)

```
sudo mkdir -p /home/deploy/var/databases/
sudo chown -R firebird:firebird /home/deploy/var/databases/
sudo chmod -R 775 /home/deploy/var/databases/
```

Verificação

```
ls -ld /home/deploy/var/databases/
namei -l /home/deploy/var/databases/
```

Output esperado

Permissões finais semelhantes a:

```
- drwxrwxr-x firebird firebird ... /home/deploy/var/databases/
```

Troubleshooting comum

- **Permission denied ao criar banco:** conferir owner/group e permissões no caminho completo.

PASSO 5: Configurar variáveis de ambiente

O que será feito

Criar `.env` no servidor com credenciais do Firebird e configurações da API.

Comandos (copiar e colar)

```
cd /home/deploy/var/www/sistema-gestao/cidadesverdes
cp .env.example .env
nano .env
```

Exemplo de `.env` (valores reais)

```
# Firebird
FIREBIRD_HOST=127.0.0.1
FIREBIRD_PORT=3050
FIREBIRD_DATABASE=/home/deploy/var/databases/cidadesverdes.fdb
FIREBIRD_USER=SYSDBA
FIREBIRD_PASSWORD=COLOQUE_SENHA_FORTE_AQUI
FIREBIRD_CHARSET=UTF8

# API Ambiente
API_HOST=0.0.0.0
API_PORT=8101
CORS_ALLOW_ORIGINS=*
```

Gerar senha segura para Firebird

```
# Opção 1 (OpenSSL)
openssl rand -base64 24

# Opção 2 (Python)
python3 - << 'PY'
import secrets
print(secrets.token_urlsafe(24))
PY
```

Ajustar segurança do arquivo

```
chmod 600 .env
ls -l .env
```

GitHub Secrets (se aplicável)

Em **GitHub** → **Settings** → **Secrets and variables** → **Actions**, cadastrar:

- FIREBIRD_HOST
- FIREBIRD_PORT
- FIREBIRD_DATABASE
- FIREBIRD_USER
- FIREBIRD_PASSWORD
- FIREBIRD_CHARSET
- API_HOST

- API_PORT
- CORS_ALLOW_ORIGINS

Verificação

```
# Conferir apenas nomes das variáveis (sem vaziar segredo)
grep -E '^(FIREBIRD_|API_|CORS_)' .env | sed 's/=.*/=*** oculto **/'
```

Troubleshooting comum

- **Variável ignorada:** conferir nome exato (case-sensitive).
- **API sobe com valores padrão:** confirmar que `.env` está no `WorkingDirectory` correto do serviço.

PASSO 6: Criar banco de dados e tabela

O que será feito

Ativar venv, instalar dependências e executar script que cria banco + schema (`ambiente` , sequence e trigger).

Comandos (copiar e colar)

```
cd /home/deploy/var/www/sistema-gestao/cidadesverdes
source .venv/bin/activate
pip install -r requirements.txt
python3 scripts/create_database.py
```

Output esperado

Mensagens como:

- Banco criado em: `/home/deploy/var/databases/cidadesverdes.fdb` (na primeira execução)
- Tabela `ambiente` criada.
- Sequence `ambiente_seq` criada.
- Trigger `bi_ambiente` criada.
- Inicialização concluída com sucesso.

Em reexecuções, é esperado aparecer `já existe`.

Verificação do banco/tabela

```
ls -lh /home/deploy/var/databases/cidadesverdes.fdb

# Abrir isql-fb e listar metadata
isql-fb -user SYSDBA -password 'SENHA_AQUI' /home/deploy/var/databases/cidades-
verdes.fdb << 'SQL'
SHOW TABLES;
SHOW SEQUENCES;
SHOW TRIGGERS;
QUIT;
SQL
```

Troubleshooting comum

- **No module named fdb:** venv não ativada ou pip instalado em ambiente errado.

- **Erro de autenticação Firebird:** revisar `FIREBIRD_USER` / `FIREBIRD_PASSWORD`.

PASSO 7: Testar a API localmente

O que será feito

Executar a API manualmente para validar endpoints antes de subir no systemd.

Comandos (copiar e colar)

```
cd /home/deploy/var/www/sistema-gestao/cidadesverdes
source .venv/bin/activate
./scripts/run_api_ambiente.sh
```

Testes de endpoint (em outro terminal SSH)

```
# Healthcheck
curl -sS http://127.0.0.1:8101/ | jq .

# Inserir
curl -sS -X POST 'http://127.0.0.1:8101/ambiente' \
  -H 'Content-Type: application/json' \
  -d '{
    "temperatura": 25.4,
    "umidade": 63.1,
    "co2": 412.7,
    "bairro": "Centro",
    "cidade": "Sao Paulo",
    "estado": "SP",
    "pais": "Brasil",
    "latitude": "-23.5505",
    "longitude": "-46.6333",
    "data": "2026-05-07",
    "hora": "10:30:00",
    "origemdaleitura": "sensor_api"
  }' | jq .

# Listar
curl -sS 'http://127.0.0.1:8101/ambiente' | jq .
```

Verificação de logs

```
# Se rodando em foreground, logs aparecem no terminal
# Caso queira guardar logs:
./scripts/run_api_ambiente.sh 2>&1 | tee /tmp/api_ambiente.log
```

Troubleshooting comum

- **Connection refused** : API não iniciou, porta errada, ou processo caiu.
 - **Erro 500**: normalmente conexão com Firebird/credencial/tabela ausente.
-

PASSO 8: Configurar serviço systemd

O que será feito

Registrar serviço para iniciar automaticamente no boot e manter API em execução.

Comandos (copiar e colar)

```
cd /home/deploy/var/www/sistema-gestao/cidadesverdes

# 1) Revisar e ajustar caminhos do arquivo de serviço
nano systemd/api-ambiente.service

# 2) Instalar serviço no systemd
sudo cp systemd/api-ambiente.service /etc/systemd/system/api-ambiente.service

# 3) Recarregar e habilitar
sudo systemctl daemon-reload
sudo systemctl enable api-ambiente

# 4) Iniciar/reiniciar
sudo systemctl restart api-ambiente

# 5) Validar status
sudo systemctl status api-ambiente --no-pager
```

Comandos de observabilidade

```
sudo journalctl -u api-ambiente -f
sudo journalctl -u api-ambiente -n 100 --no-pager
```

Output esperado

- Loaded: loaded (/etc/systemd/system/api-ambiente.service; enabled; ...)
- Active: active (running)

Troubleshooting comum

- **ExecStart inválido:** validar caminho de `uvicorn` dentro de `.venv`.
- **.env não carregado:** conferir `EnvironmentFile=` e permissões do arquivo.

PASSO 9: Liberar porta no firewall

O que será feito

Abrir a porta 8101 no UFW para acesso externo direto à API.

Comandos (copiar e colar)

```
sudo ufw allow 8101/tcp
sudo ufw status numbered
```

Output esperado

- Regra `8101/tcp ALLOW` presente.

Verificação adicional

```
sudo ss -ltnp | grep 8101 || true
```

Troubleshooting comum

- **Regra existe mas sem acesso:** validar SG/NACL da nuvem (AWS/GCP/Azure) e roteamento externo.

PASSO 10: Configurar Nginx (opcional)

O que será feito

Configurar proxy reverso para expor a API sob `location /api/ambiente/`.

Exemplo de configuração

Crie arquivo:

```
sudo nano /etc/nginx/sites-available/cidadesverdes-api
```

Conteúdo:

```
server {  
    listen 80;  
    server_name SEU_DOMINIO_OU_IP;  
  
    location /api/ambiente/ {  
        proxy_pass http://127.0.0.1:8101/;  
        proxy_http_version 1.1;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
    }  
}
```

Habilitar e aplicar:

```
sudo ln -s /etc/nginx/sites-available/cidadesverdes-api /etc/nginx/sites-enabled/cid-  
adesverdes-api  
sudo nginx -t  
sudo systemctl reload nginx
```

Verificação

```
curl -i http://127.0.0.1/api/ambiente/
```

Troubleshooting comum

- **502 Bad Gateway** : API 8101 não está ativa.
 - **Erro de sintaxe Nginx**: revisar `nginx -t`.
-

PASSO 11: Testes finais

O que será feito

Validar operação ponta a ponta (serviço ativo, acesso externo e persistência em banco).

Checklist de validação

```
# 1) Serviço ativo
sudo systemctl is-active api-ambiente

# 2) Endpoint local
curl -sS http://127.0.0.1:8101/ | jq .

# 3) Endpoint externo direto (sem Nginx)
curl -sS http://SEU_IP_PUBLICO:8101/ | jq .

# 4) Endpoint externo via Nginx (se configurado)
curl -sS http://SEU_DOMINIO/api/ambiente/ | jq .

# 5) Logs recentes da API
sudo journalctl -u api-ambiente -n 100 --no-pager

# 6) Verificar registros gravados no Firebird
isql-fb -user SYSDBA -password 'SENHA_AQUI' /home/deploy/var/databases/cidades-
verdes.fdb << 'SQL'
SELECT COUNT(*) FROM ambiente;
QUIT;
SQL
```

Comandos úteis de troubleshooting (produção)

```
# Restart seguro da API
sudo systemctl restart api-ambiente

# Reiniciar Firebird
sudo systemctl restart firebird3.0

# Status consolidado
sudo systemctl status api-ambiente firebird3.0 --no-pager

# Ver portas ativas
sudo ss -ltnp | egrep '8101|3050' || true

# Verificar variáveis no processo (debug)
sudo systemctl show api-ambiente --property=Environment
```

Critério de sucesso final

Você concluiu corretamente quando:

1. `api-ambiente` está `active (running)` ;
 2. `curl` no healthcheck retorna `{"status":"ok", ...}` ;
 3. CRUD em `/ambiente` funciona sem erro 500;
 4. Banco `.fdb` existe e recebe registros.
-

Resumo rápido (ordem de execução)

1. Commit/push no GitHub
2. `ssh servidor + git pull`
3. `./scripts/setup_firebird.sh`
4. Criar/permissions em `/home/deploy/var/databases/`
5. Criar `.env`
6. `python3 scripts/create_database.py`
7. Testar API manualmente
8. Configurar `systemd`
9. Liberar UFW 8101
10. (Opcional) Nginx `/api/ambiente/`
11. Testes finais e observabilidade